Checkpoint Report (Week 2)

1. Scope and Current Status

This Week 2 checkpoint focuses on the early progress of the project, particularly in data preparation, exploratory data analysis, baseline model training, and the initial development of the NLP and RL components for steel surface defect understanding.

At this point, the following have been completed:

- Data was acquired, cleaned, and standardized from two sources: ali2018 and neu_det.
- A reproducible train, validation, and test split was created and documented.
- An EDA notebook was prepared.
- Baseline models were trained using both a classical machine learning approach and a CNN.
- Initial CNN results and visual outputs were generated.
- A basic NLP prototype was scaffolded for text-based defect label prediction.
- An RL environment and agent stub were implemented, together with a reward design and early learning curves.
- A draft model card and an updated ethics statement were added.

2. Data Preparation Summary

The project currently uses the following preprocessing pipeline:

data/raw -> scripts/clean_dataset.py -> data/cleaned_combined -> scripts/create_splits.py -> data/splits_combined

Based on metrics/data_cleaning_summary_combined.json:

- Total images processed: **3865**
- Valid images copied: **3861**

- Duplicate images removed: **4**
- Invalid images removed: **0**

Based on metrics/split_summary_combined.json using seed = 42 and an 80/10/10 split:

- Train: **3085**
- Validation: **379**
- Test: **397**

These results show that the dataset pipeline is already organized, traceable, and reproducible, making it suitable for baseline experiments.

3. EDA and Baseline Experiments

EDA

- Notebook: notebooks/eda.ipynb
- Purpose: to examine class distribution, image characteristics, and data quality after cleaning

The EDA notebook was used to inspect the dataset before training. It includes class distribution, image dimension checks, pixel intensity distribution, and sample observations about data quality.

Baseline Metrics

The first benchmark models were evaluated using weighted metrics on the combined split.

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)
LogisticRegression (flattened grayscale)	0.5718	0.5790	0.5718	0.5501
SimpleCNN	0.8741	0.8758	0.8741	0.8641

Metric sources:

- metrics/ml_baseline_metrics_combined.json
- metrics/cnn_baseline_metrics.json
- metrics/baseline_comparison.csv

From these initial results, the CNN performs significantly better than the simple ML baseline across all reported weighted metrics.

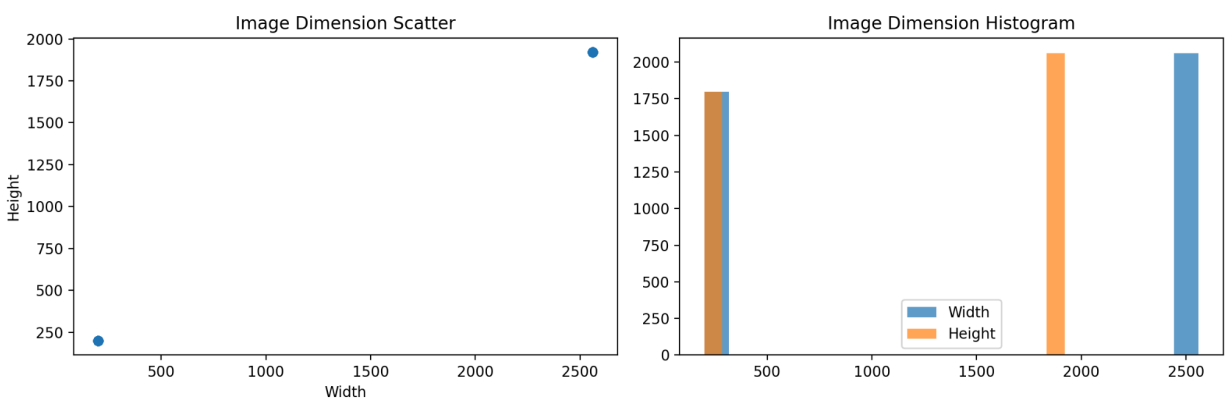
4. Visual Evidence

The following visual outputs were generated and included as evidence of progress:

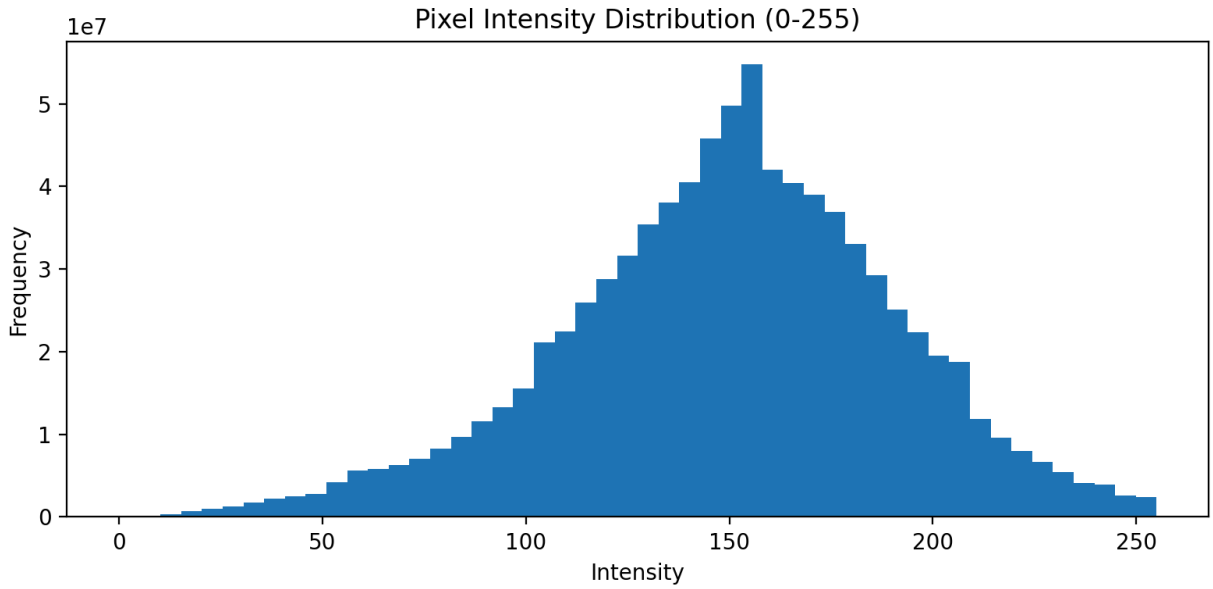
EDA figures

EDA class distribution

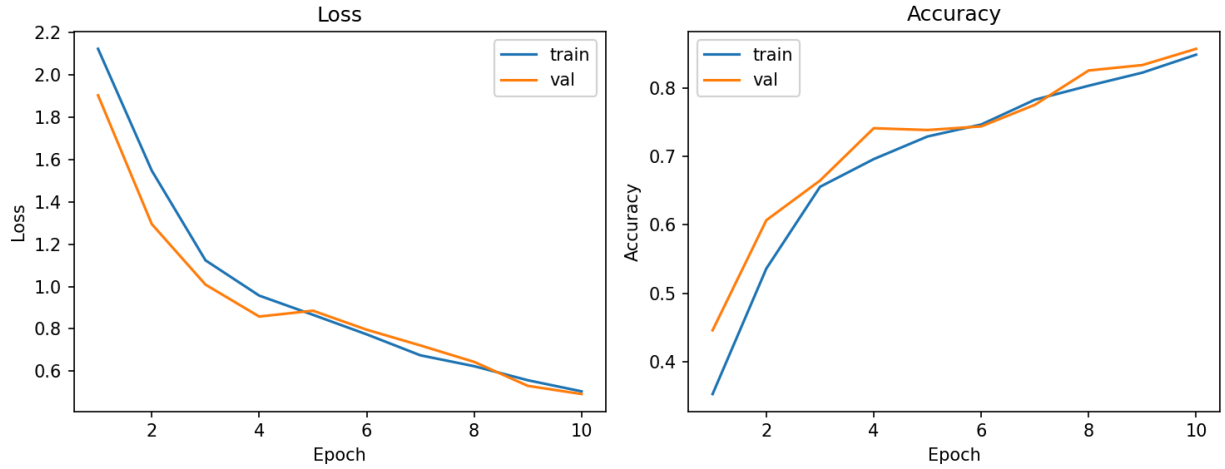
EDA image dimensions



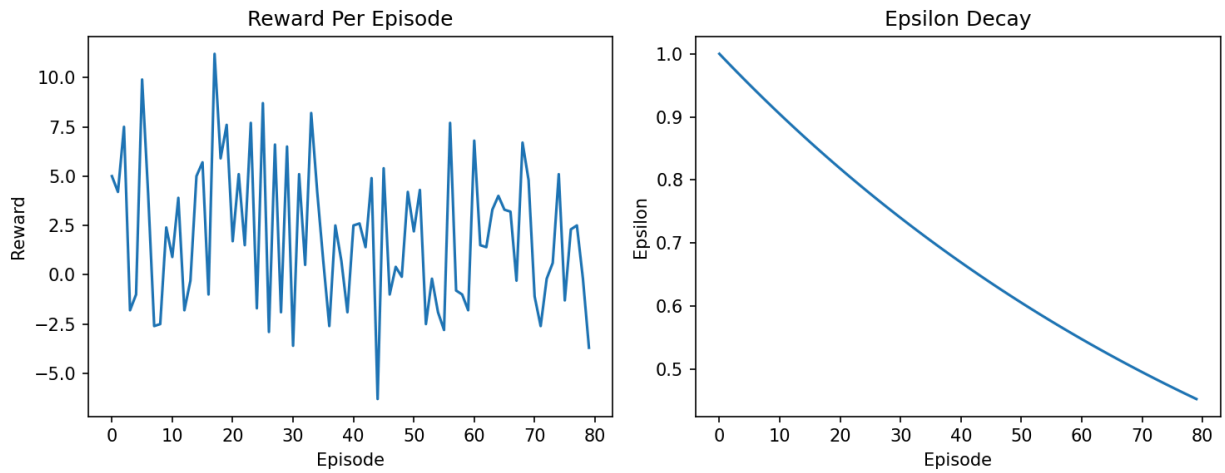
EDA pixel intensity distribution



CNN learning curves

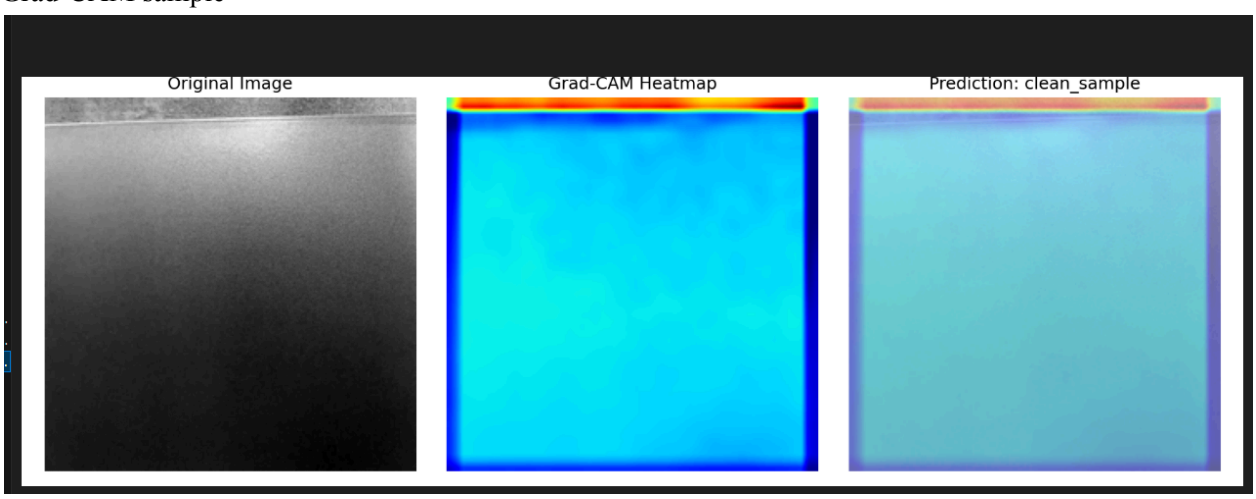


RL learning curves



Grad-CAM qualitative output:

Grad-CAM sample



Code Evidence (Key Implementation Snippets)

Data cleaning logic (scripts/clean_dataset.py)

```
if not validate_image(src):
    invalid_count += 1
    continue

file_hash = sha256_file(src)
if file_hash in seen_hashes:
    duplicate_count += 1
    continue
seen_hashes.add(file_hash)
```

CNN training flow (scripts/train_cnn_baseline.py)

```
for epoch in range(args.epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
    val_loss, val_acc, _, _ = evaluate(model, val_loader, criterion, device)

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), models_dir / "cnn_baseline_best.pth")
```

RL stub loop (rl/run_stub_experiment.py)

```
while not done:
    action = agent.choose_action()
    true_is_defect = rng.random() < 0.5
    _, reward, done = env.step(action, true_is_defect)
    ep_reward += reward
```

These snippets correspond directly to the generated metrics and plots included above.

6. NLP and RL Prototype Status

NLP Component

- File: nlp/prototype.py
- Output: metrics/nlp_prototype_output.json

Current status: the NLP prototype is already functional at a basic level using the pipeline:

clean text -> TF-IDF -> logistic regression -> label prediction

RL Component

- Files: rl/environment.py, rl/agent_stub.py, rl/run_stub_experiment.py
- Reward design: docs/reward_design.md
- Outputs: metrics/rl_stub_metrics.json, experiments/rl_learning_curves.png

Current status: the RL component is still in an early stage, but the environment, agent stub, and reward logic are already working for initial experiments. The learning curves are still noisy, which is expected for an early prototype.

7. Model Governance and Ethics

The repository already includes the following documentation:

- Draft model card: docs/model_card.md
- Updated ethics statement: docs/ethics_statement.md

These documents currently describe the intended use of the model, out-of-scope applications, known limitations, dataset-related concerns, and the need for human-in-the-loop decision-making.

8. Risks and Next Steps

At this stage, there are still several areas that need improvement:

- Class imbalance across the combined classes may reduce recall for minority classes.
- The CNN still needs more experiments, including augmentation, architecture comparison, and calibration.
- The RL setup is still a stub and does not yet represent a realistic production environment.

The next planned steps are:

1. Add class-balanced training or weighted-loss CNN training.
2. Expand evaluation through per-class error analysis and a larger Grad-CAM review set.
3. Improve the RL setup into a more stateful environment with richer observations.
4. Continue refining the model card, especially the reproducibility and failure-mode sections.